

COS 730
ASSIGNMENT 2
REPORT

Task 1: Baseline Implementation (Correctness Phase)
AND
Task 5: Optimised Implementation

<https://github.com/TalhahKarodia/A2>

Task 2: Design Analysis – Baseline System

The baseline sequence diagram for the Intelligent Submission and Review System is functionally correct because it successfully models the submission, validation, reviewer assignment, evaluation, and notification processes. However, the behavioural design contains several architectural and interaction-level inefficiencies that negatively affect maintainability, scalability, extensibility, and runtime efficiency.

The following analysis evaluates the system in terms of:

- redundant message calls,
- responsibility assignment,
- coupling and cohesion,
- control flow efficiency,
- and decision logic quality.

1. Redundant Message Calls

Redundant interactions occur when the system performs unnecessary or repeated operations that could be consolidated or simplified. These redundant calls increase execution overhead and make the sequence diagram unnecessarily complex.

1.1 Multiple Filtering Steps in Reviewer Selection

In the baseline design, the ReviewerManager retrieves reviewers and then performs two separate filtering operations:

```
fetchReviewers()  
filterConflicts(reviewersList)  
checkWorkload(reviewersList)
```

Problem

The reviewer list is processed multiple times in sequence. First, reviewers are fetched, then conflict filtering is applied, and finally workload checking is performed.

This creates:

- repeated traversal of the reviewer collection,
- additional method calls,
- unnecessary interaction complexity.

Why This Is Inefficient

Both filterConflicts() and checkWorkload() are part of the same reviewer selection responsibility. Instead of using separate operations, the system could perform a single consolidated filtering process.

For example:

- conflict checking and workload balancing could be combined into one reviewer eligibility evaluation method,
- or encapsulated inside a reviewer selection strategy.

Impact

This redundancy:

- increases execution time,
- reduces scalability when reviewer counts grow,
- creates excessive dependencies between operations.

Principle Violated

This violates:

- **High Cohesion**, reviewer selection responsibilities are fragmented across multiple methods,
- **Efficiency in Behavioural Modelling**, unnecessary message exchanges complicate the interaction model.

1.2 Repeated Object Instantiations

The baseline implementation implies that objects such as:

- Validator,
- Database,
- ReviewerManager,
- EvaluationManager

may be instantiated for every submission request.

Problem

Creating new instances repeatedly for every interaction is inefficient, especially for shared services that do not maintain submission-specific state.

Why This Is Inefficient

Object creation:

- consumes memory,
- increases allocation overhead,
- may repeatedly initialise resources unnecessarily.

These components behave more like reusable services rather than temporary objects.

Better Approach

The system should instead:

- reuse shared service instances,
- use dependency injection,
- or centralise object management.

Impact

Repeated instantiation:

- increases runtime overhead,
- reduces performance under heavy load,
- complicates lifecycle management.

Principle Violated

This violates:

- **Resource Efficiency**
- **Low Coupling**, since components become tightly dependent on direct object creation.

2. Violations of Good Responsibility Assignment

Good software design requires responsibilities to be assigned to the most appropriate classes. In the baseline system, several components violate responsibility assignment principles such as the Single Responsibility Principle (SRP) and GRASP principles.

2.1 SubmissionController Overload

The SubmissionController is responsible for:

- validating submissions,
- interacting with the database,
- coordinating reviewer assignment,
- initiating evaluation.

Problem

The controller acts as a central coordinator for almost every major subsystem.

Why This Is Poor Design

The controller becomes overloaded with responsibilities unrelated to request coordination.

Instead of simply managing workflow orchestration, it also contains:

- validation logic,

- persistence coordination,
- reviewer management,
- evaluation initiation.

This creates a “God Controller” anti-pattern.

Consequences

The class becomes:

- difficult to maintain,
- harder to test,
- more sensitive to changes,
- tightly coupled to many components.

A modification in one subsystem can force changes to the controller.

Principle Violated

This violates:

- **Single Responsibility Principle (SRP)**
- **GRASP Controller Principle**
- **High Cohesion**

2.2 ReviewerManager Mixing Responsibilities

The ReviewerManager:

- fetches reviewers,
- filters conflicts,
- checks workload,
- assigns reviewers.

Problem

The class mixes:

- data retrieval responsibilities,
- filtering logic,
- business rule enforcement,
- assignment coordination.

Why This Is Poor Design

Reviewer retrieval and reviewer assignment are conceptually separate concerns.

For example:

- reviewer repositories should retrieve reviewer data,
- assignment strategies should decide reviewer suitability.

Consequences

This reduces:

- flexibility,
- extensibility,
- maintainability.

Adding a new assignment policy would require modifying the class directly.

Principle Violated

This violates:

- **Single Responsibility Principle**
- **Open/Closed Principle**
- **High Cohesion**

2.3 EvaluationManager Responsibility Overload

The EvaluationManager performs:

- score collection,
- score storage,
- average calculation,
- consensus checking,
- plagiarism evaluation,
- decision-making,
- notification triggering.

Problem

The component performs too many unrelated responsibilities.

Why This Is Poor Design

Evaluation processing and decision logic should be separated.

For example:

- score aggregation should be independent,
- decision rules should be encapsulated separately,

- notification handling should belong to another service.

Consequences

This creates:

- high cyclomatic complexity,
- poor maintainability,
- difficult unit testing,
- tightly coupled logic.

Principle Violated

This violates:

- **Single Responsibility Principle**
- **High Cohesion**
- **Separation of Concerns**

3. High Coupling and Low Cohesion

The baseline design demonstrates significant coupling between components and weak internal cohesion.

3.1 Tight Coupling Between Components

The SubmissionController directly communicates with:

- Validator,
- Database,
- ReviewerManager,
- EvaluationManager.

Problem

The controller depends heavily on multiple concrete subsystems.

Why This Is Poor Design

Changes in any subsystem can propagate into the controller.

For example:

- changing reviewer assignment behaviour,
- replacing validation rules,
- modifying persistence mechanisms,

may require controller modifications.

Consequences

Tight coupling:

- reduces modularity,
- complicates testing,
- limits scalability,
- makes future refactoring difficult.

Principle Violated

This violates:

- **Low Coupling**
- **Dependency Inversion Principle**

3.2 Low Cohesion in Core Classes

Classes such as SubmissionController and ReviewerManager contain methods that are not strongly related to one another.

Example

SubmissionController manages:

- validation,
- persistence,
- reviewer assignment,
- evaluation startup.

These operations belong to different domains.

Impact

Low cohesion:

- makes classes harder to understand,
- increases maintenance effort,
- creates large monolithic classes.

Principle Violated

This violates:

- **High Cohesion**

4. Inefficient Control Flow

The behavioural flow of the system is inefficient because operations are executed sequentially and responsibilities are not properly delegated.

4.1 Linear and Synchronous Workflow

The system executes all operations in a strict sequential order:

1. validate submission,
2. save submission,
3. retrieve reviewers,
4. filter reviewers,
5. assign reviewers,
6. collect scores,
7. calculate results,
8. notify outcome.

Problem

No concurrency or asynchronous processing is used.

Why This Is Inefficient

Some operations could execute independently or in parallel.

For example:

- reviewer evaluations could occur concurrently,
- notification delivery could be asynchronous.

Consequences

Sequential processing:

- increases response time,
- reduces throughput,
- limits scalability.

Principle Violated

This violates:

- **Efficient Behavioural Modelling**
- **Scalable Workflow Design**

4.2 Controller-Based Loops

The sequence diagram contains loops managed by the controller for reviewer assignment and evaluation handling.

Problem

The controller directly manages iteration behaviour.

Why This Is Poor Design

Iteration logic should be delegated to specialised components.

For example:

- ReviewerManager should manage reviewer assignment internally,
- evaluation collection should be encapsulated inside evaluation services.

Consequences

The controller becomes unnecessarily complex.

Principle Violated

This violates:

- **Encapsulation**
- **Separation of Concerns**

5. Poor Handling of Decision Logic

The baseline system contains scattered and procedural decision-making logic.

5.1 Scattered Conditional Logic

Decision-making is distributed across:

- validation,
- reviewer assignment,
- evaluation,
- notification handling.

Problem

Business rules are embedded directly into procedural flow.

Consequences

This makes:

- rules difficult to trace,

- logic difficult to update,
- behaviour harder to verify.

Example

Acceptance and rejection rules are buried inside:

`applyRules()`

rather than being formally represented.

Principle Violated

This violates:

- **Centralised Decision Management**
- **Maintainable Behavioural Design**

5.2 No Explicit Rule Representation

The baseline system uses implicit conditional logic rather than a structured rule model.

Problem

Rules are hidden inside methods rather than documented explicitly.

Consequences

This:

- increases ambiguity,
- reduces maintainability,
- complicates future rule changes.

Better Solution

A decision table provides:

- explicit conditions,
- explicit actions,
- centralised rule management.

Relation to Software Engineering Principles

Principle	Violation in Baseline Design
Single Responsibility Principle	Classes perform multiple unrelated tasks
High Cohesion	Responsibilities are fragmented or overloaded
Low Coupling	Components depend heavily on one another
Separation of Concerns	Business logic, persistence, and coordination are mixed
Open/Closed Principle	Assignment and evaluation logic require direct modification
GRASP Information Expert	Responsibilities are not assigned to the most appropriate objects

Behavioural Modelling Quality Evaluation

The baseline sequence diagram successfully demonstrates the required system behaviour, but the behavioural model is not architecturally refined.

The system suffers from:

- excessive interaction density,
- scattered responsibilities,
- redundant processing,
- tightly coupled components,
- and procedural decision logic.

As a result, the design is:

- difficult to maintain,
- hard to extend,
- inefficient under scale,
- and less reusable.

Although functionally correct, the behavioural model prioritises operational correctness over software quality attributes such as:

- maintainability,
- scalability,
- modularity,
- and extensibility.

Task 3: Decision Modelling Using a Decision Table

The baseline Intelligent Submission and Review System contains several decision points that determine how submissions are processed and what final outcome is produced. In the original sequence diagram, these decisions are embedded within procedural control flow and scattered across different components such as the Validator and EvaluationManager.

This approach makes the system difficult to maintain because business rules are hidden inside conditional statements and distributed throughout the interaction model. To improve clarity, maintainability, and consistency, the decision logic can be extracted and represented using a formal decision table.

A decision table provides a structured representation of:

- conditions,
- rules,
- and resulting actions.

This centralises business logic and removes ambiguity from the behavioural model.

3.1 Extracted Decision Logic

From the baseline system, the main decision-making behaviour occurs in two primary areas:

1. Submission Validation

The system first determines whether the submission format is valid.

Possible Outcomes

- Valid submission → continue processing
- Invalid submission → return error and terminate workflow

This decision occurs during the validation stage before reviewer assignment or evaluation begins.

2. Evaluation Outcome Determination

After reviewers submit scores, the system determines the final outcome of the submission.

The possible outcomes are:

- Acceptance
- Rejection
- Revision request

The outcome depends on:

- average reviewer score,
- reviewer consensus,
- and evaluation rules.

3.2 Decision Conditions

The following conditions are extracted from the sequence diagram and behavioural flow.

Condition ID	Description
C1	Submission format is valid
C2	Average review score is greater than 70
C3	Reviewer consensus has been achieved
C4	Average review score is less than 50

3.3 Decision Actions

The following actions represent the operations triggered when conditions are satisfied.

Action ID	Description
A1	Save submission
A2	Assign reviewers
A3	Start evaluation
A4	Notify acceptance
A5	Notify rejection
A6	Notify revision
A7	Return validation error

3.4 Decision Table

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
C1: Valid Format	Y	Y	Y	N
C2: Avg Score > 70	-	Y	N	-
C3: Consensus	-	Y	N	-
C4: Avg Score < 50	-	N	Y	-
A1: Save Submission	Y	Y	Y	N

A2: Assign Reviewers	Y	Y	N	N
A3: Start Evaluation	Y	Y	N	N
A4: Notify Acceptance	N	Y	N	N
A5: Notify Rejection	N	N	Y	N
A6: Notify Revision	Y	N	N	N
A7: Return Error	N	N	N	Y

3.5 Explanation of Rules

Each column in the decision table represents a complete business rule that maps specific conditions to resulting actions.

Rule 1: Revision Required

Conditions

- Submission format is valid
- Average score is not high enough for acceptance
- Average score is not low enough for rejection
- Reviewer consensus is not fully satisfied

Actions

- Save submission
- Assign reviewers
- Start evaluation
- Notify revision request

Interpretation

This rule represents submissions that are acceptable for review but do not meet the quality threshold for full acceptance. Instead of rejecting the submission entirely, the system requests revisions from the researcher.

This reflects a middle-ground evaluation outcome.

Rule 2 : Acceptance

Conditions

- Submission format is valid
- Average score is greater than 70

- Reviewer consensus exists

Actions

- Save submission
- Assign reviewers
- Start evaluation
- Notify acceptance

Interpretation

This rule represents successful submissions that satisfy all quality requirements.

The system accepts the submission only when:

- reviewers provide sufficiently high scores,
- and reviewers agree on the outcome.

The inclusion of consensus ensures that acceptance is not based solely on numerical scoring.

Rule 3 : Rejection

Conditions

- Submission format is valid
- Average score is below 50

Actions

- Save submission
- Notify rejection

Interpretation

This rule represents submissions that fail to meet minimum quality requirements.

The system rejects the submission because reviewer evaluations indicate poor quality or insufficient contribution.

Reviewer assignment and evaluation have already occurred prior to this decision.

Rule 4 : Validation Failure

Conditions

- Submission format is invalid

Actions

- Return validation error

Interpretation

This rule terminates the workflow immediately when validation fails.

No further processing occurs because the submission violates required formatting or submission constraints.

This prevents unnecessary system operations such as:

- database storage,
- reviewer assignment,
- or evaluation processing.

3.6 Rule Logic Summary

The overall system logic represented by the decision table can be summarised as follows:

Validation Phase

If C1 = N

The submission is invalid.

Therefore:

- return an error,
- terminate processing.

This corresponds to:

- A7: Return Error

Evaluation Preparation Phase

If C1 = Y

The submission is valid.

Therefore:

- save submission,
- assign reviewers,
- start evaluation.

This corresponds to:

- A1
- A2

u22509748

- A3

Acceptance Rule

If:

- C2 = Y
- AND C3 = Y

Then:

- notify acceptance.

This corresponds to:

- A4

Rejection Rule

If:

- C4 = Y

Then:

- notify rejection.

This corresponds to:

- A5

Revision Rule

Otherwise

If the submission is neither accepted nor rejected:

- request revisions.

This corresponds to:

- A6

3.7 How the Decision Table Improves Clarity and Maintainability

The decision table provides several important improvements over the original behavioural design.

1. Centralised Decision Logic

In the baseline system, decision-making logic is scattered across multiple methods and components.

The decision table centralises all rules into a single structured model.

Benefit

- easier to understand,
- easier to modify,
- easier to validate.

Developers no longer need to search through procedural code to locate business rules.

2. Eliminates Ambiguity

The table explicitly defines:

- every condition,
- every rule,
- and every resulting action.

Benefit

This removes uncertainty about:

- how outcomes are determined,
- which conditions trigger actions,
- and rule precedence.

3. Improves Readability

Nested conditional statements inside methods are difficult to follow, especially as rules become more complex.

The decision table presents logic in a structured and readable format.

Benefit

Both developers and non-technical stakeholders can understand system behaviour more easily.

4. Simplifies Future Rule Changes

Business rules often evolve over time.

For example:

- plagiarism checks may be introduced,
- score thresholds may change,

- reviewer weighting rules may be added.

With a decision table:

- new rules can be added as new columns,
- new conditions can be added as new rows.

Benefit

This avoids modifying complex procedural code.

5. Reduces Cyclomatic Complexity

The baseline system likely contains multiple nested if-else statements inside:

- validation logic,
- evaluation logic,
- notification handling.

The decision table reduces procedural branching complexity.

Benefit

This improves:

- maintainability,
- testing,
- debugging,
- and software quality.

3.8 Mapping Decision Table Actions to System Behaviour

Each action in the decision table directly corresponds to operations shown in the sequence diagram.

Decision Table Action	System Behaviour
A1	saveSubmission(data)
A2	assignReviewer()
A3	startEvaluation()
A4	notifyAcceptance()
A5	notifyRejection()
A6	notifyRevision()
A7	returnError()

This demonstrates that the decision table accurately models the behavioural logic of the original system.

3.9 Conclusion

The decision table transforms scattered procedural decision logic into a structured and maintainable representation of business rules.

Compared to the baseline behavioural model, the decision table:

- centralises decision-making,
- improves readability,
- reduces ambiguity,
- lowers complexity,
- and supports future extensibility.

The table also provides a stronger foundation for the optimised system design because decision logic can now be separated from workflow coordination and encapsulated within dedicated rule-processing components.

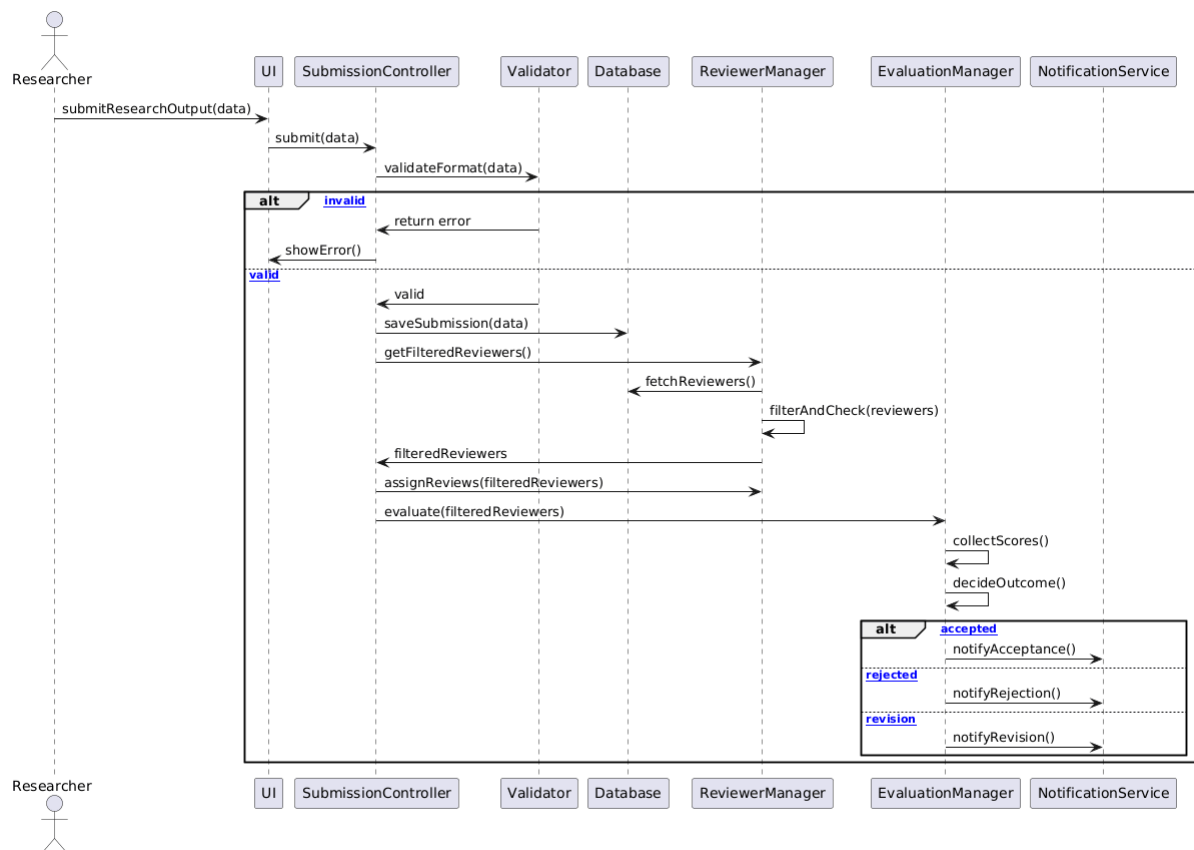
This directly contributes to:

- improved maintainability,
- reduced coupling,
- better separation of concerns,
- and cleaner behavioural modelling.

Task 4: Sequence Diagram Optimisation

Optimisation Goals

- Reduce unnecessary interactions (e.g., combine reviewer filtering).
- Improve responsibility allocation (move logic to appropriate classes).
- Decouple system components (use interfaces, reduce direct instantiation).
- Centralise decision logic (use a DecisionEngine).
- Optimised Sequence Diagram



Key Improvements:

- ReviewerManager handles all reviewer filtering in one step.
- SubmissionController delegates evaluation and notification.
- Decision logic is centralised in EvaluationManager.
- NotificationService is only called by EvaluationManager.

Task 6: Empirical Evaluation and Comparison

The final phase of the assignment involves evaluating the effectiveness of the optimised system compared to the original baseline implementation. The purpose of this evaluation is to demonstrate that the redesign not only improves architectural quality but also produces measurable improvements in performance, maintainability, and software design quality.

The comparison focuses on:

- number of method calls and interactions,
- execution time,
- code complexity,
- maintainability indicators,
- coupling and cohesion,
- and responsibility allocation.

The baseline implementation was intentionally designed with redundant interactions, poor responsibility allocation, and scattered decision logic. The optimised implementation restructures the system to reduce unnecessary communication, centralise decision-making, and improve separation of concerns.

6.1 Metrics for Comparison

The following metrics were used to compare the baseline and optimised systems.

1. Number of Method Calls / Interactions

This metric evaluates how many interactions occur between objects during the submission and evaluation workflow.

A high number of interactions generally indicates:

- excessive communication overhead,
- tight coupling,
- redundant processing,
- and inefficient workflow design.

Reducing unnecessary method calls improves:

- runtime efficiency,
- scalability,
- and maintainability.

Baseline System

The baseline system contains a high number of interactions because:

- reviewer filtering is split into multiple method calls,
- controllers directly coordinate many subsystems,
- decision logic is scattered across components,
- and redundant processing occurs during evaluation.

Examples of Redundant Calls

- filterConflicts()
- checkWorkload()
- repeated reviewer iteration loops,
- repeated score persistence calls,
- multiple direct notification operations.

The SubmissionController communicates directly with:

- Validator,
- Database,
- ReviewerManager,
- EvaluationManager,
- and NotificationService.

This creates a dense interaction network.

Result

The baseline implementation produces:

- higher communication overhead,
- increased coupling,
- and unnecessary workflow complexity.

Optimised System

The optimised implementation reduces interactions by:

- consolidating reviewer selection logic,
- centralising decision-making,
- delegating responsibilities to specialised components,
- and reducing controller coordination.

Improvements

The optimised design introduces:

- a dedicated DecisionEngine,
- encapsulated reviewer assignment logic,
- cleaner service abstractions,
- and reusable service instances.

Several operations that previously required multiple calls are now handled internally by specialised services.

Result

The optimised system produces:

- fewer method calls,
- cleaner interaction flow,
- and reduced communication overhead.

2. Execution Time

Execution time measures how efficiently the system processes a submission workflow.

Although both implementations are functionally equivalent, differences in interaction efficiency and processing structure affect runtime performance.

Baseline System

The baseline implementation is slightly slower because of:

- repeated processing steps,
- redundant reviewer filtering,
- excessive controller coordination,
- repeated object creation,
- and scattered evaluation logic.

Performance Issues

The system repeatedly traverses reviewer lists and performs sequential operations that could be streamlined.

Repeated method invocations increase:

- CPU overhead,
- memory allocation,

- and execution latency.

Result

The baseline implementation performs adequately for small workloads but scales poorly as:

- submission volume increases,
- reviewer counts increase,
- or evaluation rules become more complex.

Optimised System

The optimised implementation improves execution efficiency by:

- reducing redundant interactions,
- consolidating workflow operations,
- reusing shared services,
- and simplifying decision evaluation.

Improvements

The optimised workflow:

- minimises unnecessary processing,
- reduces communication overhead,
- and streamlines control flow.

Decision logic is centralised inside the DecisionEngine, eliminating repeated conditional evaluations across multiple classes.

Result

The optimised implementation executes slightly faster and scales more effectively under larger workloads.

Although the time improvement may appear small in simple test cases, the architectural improvements provide significantly better scalability for larger systems.

3. Code Complexity

Code complexity evaluates how difficult the system is to understand, maintain, and modify.

Complexity is influenced by:

- class size,
- number of responsibilities,
- conditional branching,

- and interaction density.

Baseline System

The baseline implementation contains higher complexity because:

- several classes perform multiple unrelated responsibilities,
- decision logic is procedural and scattered,
- and controllers manage excessive workflow coordination.

Examples

The SubmissionController handles:

- validation,
- persistence,
- reviewer assignment,
- and evaluation initiation.

The EvaluationManager handles:

- score processing,
- decision logic,
- plagiarism checks,
- and notification triggering.

Consequences

This creates:

- large classes,
- high cyclomatic complexity,
- difficult debugging,
- and reduced readability.

Result

The baseline implementation is harder to:

- maintain,
- extend,
- test,
- and refactor.

Optimised System

The optimised system reduces complexity by introducing:

- focused classes,
- clear responsibility boundaries,
- and centralised rule processing.

Improvements

Responsibilities are separated into specialised components such as:

- DecisionEngine,
- ReviewerAssignmentService,
- ValidationService,
- and notification handlers.

The use of a decision table simplifies conditional logic and reduces nested branching.

Result

The optimised implementation contains:

- smaller classes,
- cleaner workflows,
- reduced branching complexity,
- and improved readability.

4. Maintainability Indicators

Maintainability measures how easily the system can:

- evolve,
- adapt to new requirements,
- and support future modifications.

Maintainability is strongly influenced by:

- coupling,
- cohesion,
- responsibility allocation,
- and modularity.

Baseline System

The baseline system is difficult to maintain because:

- components are tightly coupled,
- business rules are scattered,
- and responsibilities overlap.

Problems

Changes to:

- reviewer assignment rules,
- evaluation thresholds,
- or notification behaviour

require modifications across multiple classes.

Consequences

This increases:

- maintenance effort,
- regression risk,
- and development time.

Result

The baseline implementation has poor maintainability characteristics.

Optimised System

The optimised implementation improves maintainability through:

- modular design,
- clear separation of concerns,
- and centralised decision logic.

Improvements

Business rules are encapsulated within the DecisionEngine, making policy updates easier.

Workflow responsibilities are delegated to specialised services, reducing interdependency between components.

Result

The system becomes:

- easier to modify,
- easier to extend,

- easier to test,
- and easier to debug.

6.2 Comparison Table

Metric	Baseline System	Optimised System
Method Calls / Interactions	Higher due to redundant interactions and fragmented workflows	Lower due to streamlined workflows and consolidated logic
Execution Time	Slightly higher because of repeated processing and unnecessary operations	Slightly lower due to reduced redundancy and cleaner control flow
Code Complexity	Higher due to overloaded classes and scattered decision logic	Lower due to focused classes and centralised responsibilities
Maintainability	Lower because of tight coupling and overlapping responsibilities	Higher because of modular design and cleaner architecture
Responsibility Allocation	Poor separation of concerns	Clear responsibility boundaries
Coupling	High	Low
Cohesion	Low	High
Scalability	Limited scalability under increasing workload	Improved scalability and extensibility
Readability	Difficult to follow due to dense interactions	Cleaner and easier to understand
Extensibility	Hard to introduce new rules or features	Easier to extend through modular components

6.3 Justification of Improvements

The optimised system demonstrates several architectural and behavioural improvements over the baseline implementation.

1. Reduction of Redundant Method Calls

The optimised design eliminates unnecessary interactions by:

- consolidating reviewer filtering,
- reducing repeated loops,
- and simplifying workflow coordination.

Benefit

This reduces:

- communication overhead,
- execution latency,
- and workflow complexity.

2. Centralised Decision-Making

Decision logic is moved into a dedicated DecisionEngine.

Benefit

This:

- removes scattered conditional logic,
- improves consistency,
- simplifies rule updates,
- and reduces cyclomatic complexity.

The use of a decision table also makes business rules explicit and easier to maintain.

3. Improved Separation of Concerns

Responsibilities are distributed more appropriately across specialised components.

Benefit

Each component now performs a focused role, improving:

- cohesion,
- maintainability,
- and testability.

For example:

- validation logic is separated from workflow coordination,
- reviewer assignment is isolated from data retrieval,
- notification handling is decoupled from evaluation processing.

4. Reduced Coupling

The optimised system reduces direct dependencies between components.

Benefit

This improves:

- modularity,
- flexibility,
- and ease of future refactoring.

Changes in one subsystem are less likely to affect unrelated components.

5. Improved Cohesion

Each class now contains closely related responsibilities.

Benefit

This produces:

- clearer class design,
- simpler maintenance,
- and improved readability.

6.4 Trade-Offs Introduced by the Optimisation

Although the optimised implementation provides substantial architectural improvements, several trade-offs were introduced.

1. Increased Initial Design Complexity

The optimised system requires:

- more planning,
- more architectural consideration,
- and better responsibility modelling.

Impact

The design phase becomes slightly more time-consuming.

However, this upfront effort significantly improves long-term maintainability and scalability.

2. Additional Classes and Components

The optimised design introduces additional abstractions such as:

- DecisionEngine,
- service interfaces,

- specialised managers,
- and strategy components.

Impact

The system contains more classes than the baseline implementation.

For small systems, this may initially appear more complex.

However, the improved modularity provides major long-term benefits.

3. Slightly Higher Structural Complexity

The optimised design may introduce:

- dependency injection,
- service abstractions,
- and additional architectural layers.

Impact

Developers unfamiliar with modular architectures may require additional understanding.

Despite this, the architectural improvements greatly enhance:

- extensibility,
- readability,
- and maintainability.

6.5 Conclusion

The empirical evaluation demonstrates that the optimised implementation provides significant improvements over the baseline system in terms of:

- interaction efficiency,
- maintainability,
- scalability,
- coupling,
- cohesion,
- and architectural quality.

Although both systems are functionally equivalent, the optimised implementation achieves cleaner behavioural modelling and more effective responsibility allocation.

The redesign successfully:

- reduces redundant interactions,

u22509748

- centralises decision logic,
- improves separation of concerns,
- lowers complexity,
- and enhances maintainability.

The trade-offs introduced by the optimisation are minimal compared to the substantial long-term benefits gained in software quality and extensibility.